

Simulating Logistics Systems

Lesson 10/15

João Flávio de Freitas Almeida

LEPOINT: Laboratório de Estudos em Planejamento de Operações Integradas
Departamento de Engenharia de Produção

Universidade Federal de Minas Gerais
Escola de Engenharia

UFMG, Belo Horizonte, Brasil
15.10.2025

Outline of this lecture

ACD Simpy Models

SimPy Project

Outline of this lecture

ACD Simpy Models

SimPy Project

SimPy: PriorityResource [SimPy Contributors, 2025]

```
import simpy

def pacientePri(env, nome, prio, medicoCirurgiao):
    print(f'{env.now}: {nome} (prio: {prio}) chega.')
    with medicoCirurgiao.request(priority=prio) as req:
        yield req # O paciente aguarda recurso
        print(f'{env.now}: {nome} (prio: {prio}) eh atendido')
        yield env.timeout(5)
        print(f'{env.now}: {nome} (prio: {prio}) finaliza o
              atendimento para o processo de cirurgia')
```

SimPy: PriorityResource [SimPy Contributors, 2025]

```
def geraPacientesPrio(env, medicoCirurgiao):
    prios = [0, 1, 2, 3, 4] # Prioridades dos pacientes
    # prios = [4, 1, 0, 3, 2] # Retirar comentario e testar
    for i in range(5):
        prio = prios[i % len(prios)] # Rotaciona prioridades
        env.process(pacientePri(env, f'Paciente {i+1}', prio,
                                medicoCirurgiao))
        # yield env.timeout(3) # Intervalo fixo entre chegadas
        : Pacientes chegam a cada 3 unidades de tempo
        yield env.timeout(0) # Supondo que todos pacientes
        chegam ao mesmo tempo (simplificacao)

env = simpy.Environment()
medicoCirurgiao = simpy.PriorityResource(env, capacity=1)
env.process(geraPacientesPrio(env, medicoCirurgiao))
env.run(until=30)
```

SimPy: PriorityResource [SimPy Contributors, 2025]

```
● (venv) PS C:\Users\user\Desktop\s1s\08> py .\ex1-priority-resource.py
0: Paciente 1 (prioridade: 0) chega para cirurgia.
0: Paciente 2 (prioridade: 1) chega para cirurgia.
0: Paciente 1 (prio: 0) começa a ser atendido no processo de cirurgia
0: Paciente 3 (prioridade: 2) chega para cirurgia.
0: Paciente 4 (prioridade: 3) chega para cirurgia.
0: Paciente 5 (prioridade: 4) chega para cirurgia.
5: Paciente 1 (prio: 0) finaliza o atendimento para o processo de cirurgia
5: Paciente 2 (prio: 1) começa a ser atendido no processo de cirurgia
10: Paciente 2 (prio: 1) finaliza o atendimento para o processo de cirurgia
10: Paciente 3 (prio: 2) começa a ser atendido no processo de cirurgia
15: Paciente 3 (prio: 2) finaliza o atendimento para o processo de cirurgia
15: Paciente 4 (prio: 3) começa a ser atendido no processo de cirurgia
20: Paciente 4 (prio: 3) finaliza o atendimento para o processo de cirurgia
20: Paciente 5 (prio: 4) começa a ser atendido no processo de cirurgia
25: Paciente 5 (prio: 4) finaliza o atendimento para o processo de cirurgia
```

SimPy: PreemptiveResource [SimPy Contributors, 2025]

```
def pacPreemp(env, nome, prio, SAMU):
    tempRestante = 5 # Tempo de atendimento necessario
    while tempRestante > 0:
        with SAMU.request(priority=prio, preempt=True) as req:
            t_inicio = None # Inicializa a variavel
            try:
                yield req
                t_inicio = env.now
                yield env.timeout(tempRestante) #Tenta finalizar
                tempRestante = 0 # Atendimento concluido
            except simpy.Interrupt as Int:
                pacInt = "paciente de maior prioridade"
                if hasattr(Int.cause, 'by') and Int.cause.by:
                    try:
                        prioInt = Int.cause.by.priority
                        pacInt = f"paciente de prioridade {
                            prioInt}"
                    except:
                        pass
```

SimPy: PreemptiveResource [SimPy Contributors, 2025]

```
        if t_inicio is not None: # Houve interrupcao
            tempo_decorrido = env.now - t_inicio
            tempRestante -= tempo_decorrido
        else:
            # Interrup antes atend. Tentara novamente

def geraPacientesPreemptivos(env, SAMU):
    # prios = [0, 1, 2, 3, 4] # Prioridades dos pacientes
    prios = [4, 1, 0, 3, 2] # Prioridades dos pacientes

    for i in range(1,6):
        prio = prios[i % len(prios)] # Rotaciona prioridades
        env.process(pacPreemp(env, f'Pac {i}', prio, SAMU))
        yield env.timeout(0) # (simplificar)

env = simpy.Environment()
SAMU = simpy.PreemptiveResource(env, capacity=1)
env.process(geraPacientesPreemptivos(env, SAMU))
env.run(until=30)
```


SimPy: PreemptiveResource [SimPy Contributors, 2025]

```
• (venv) PS C:\Users\user\Desktop\sls\08> py .\ex2-preemptive-resource.py
0: Paciente 1 (prioridade: 4) pede atendimento ao SAMU.
0: Paciente 2 (prioridade: 1) pede atendimento ao SAMU.
0: Paciente 1 (prio: 4) foi interrompido por paciente de maior prioridade. Tentará novamente.
0: Paciente 3 (prioridade: 0) pede atendimento ao SAMU.
0: Paciente 2 (prio: 1) foi interrompido por paciente de maior prioridade. Tentará novamente.
0: Paciente 4 (prioridade: 3) pede atendimento ao SAMU.
0: Paciente 3 (prio: 0) recebe atendimento na ambulância do SAMU.
0: Paciente 5 (prioridade: 2) pede atendimento ao SAMU.
5: Paciente 3 (prio: 0) finaliza atendimento na ambulância do SAMU.
5: Paciente 2 (prio: 1) recebe atendimento na ambulância do SAMU.
10: Paciente 2 (prio: 1) finaliza atendimento na ambulância do SAMU.
10: Paciente 5 (prio: 2) recebe atendimento na ambulância do SAMU.
15: Paciente 5 (prio: 2) finaliza atendimento na ambulância do SAMU.
15: Paciente 4 (prio: 3) recebe atendimento na ambulância do SAMU.
20: Paciente 4 (prio: 3) finaliza atendimento na ambulância do SAMU.
20: Paciente 1 (prio: 4) recebe atendimento na ambulância do SAMU.
25: Paciente 1 (prio: 4) finaliza atendimento na ambulância do SAMU.
```

```
import simpy

def solicitaRemedios(env, estFarm):
    est_min = 50
    pedidos = [15, 30, 45]  # Diferentes qdes
    for i in range(1,11):
        yield env.timeout(5)  #Ag 5 min entre pedidos
        ped = pedidos[i % len(pedidos)]  # Rotaciona qdes
        if estFarm.level >= est_min:
            estFarm.get(ped)
        else:
            env.process(reabasteceEst1Vez(env, 40, estFarm))

def reabasteceEst(env, qde, estFarm):
    while True:
        yield env.timeout(20)  # Ag. 20 min entre reabastec.
        estFarm.put(qde)  # Adiciona a qde ao estoque
```

```
def reabasteceEst1Vez(env, qde, estFarm):  
    yield env.timeout(10) #Ag 10 min entre reabastecimentos  
    estFarm.put(qde) #Adiciona a qde ao estoque  
  
env = simpy.Environment()  
estFarm = simpy.Container(env, init=100, capacity=200)  
env.process(solicitaRemedios(env, estFarm))  
env.process(reabasteceEstoque(env, 50, estFarm))  
env.run(until=100)
```

SimPy: Containers [SimPy Contributors, 2025]

```
(venv) PS C:\Users\user\Desktop\sfs\08> py .\ex3-containers.py
5: Pedido 1: Solicitou 30 unidades. Nível atual do estoque: 70
10: Pedido 2: Solicitou 45 unidades. Nível atual do estoque: 25
15: Pedido 3: Tentou solicitar 15 unidades, mas estoque insuficiente (25).
20: (Rotina) Reabasteceu 50 unidades. Nível atual do estoque: 75
20: Pedido 4: Solicitou 30 unidades. Nível atual do estoque: 45
25: Reabasteceu 40 unidades. Nível atual do estoque: 85
25: Pedido 5: Solicitou 45 unidades. Nível atual do estoque: 40
30: Pedido 6: Tentou solicitar 15 unidades, mas estoque insuficiente (40).
35: Pedido 7: Tentou solicitar 30 unidades, mas estoque insuficiente (40).
40: (Rotina) Reabasteceu 50 unidades. Nível atual do estoque: 90
40: Reabasteceu 40 unidades. Nível atual do estoque: 130
40: Pedido 8: Solicitou 45 unidades. Nível atual do estoque: 85
45: Reabasteceu 40 unidades. Nível atual do estoque: 125
45: Pedido 9: Solicitou 15 unidades. Nível atual do estoque: 110
50: Pedido 10: Solicitou 30 unidades. Nível atual do estoque: 80
60: (Rotina) Reabasteceu 50 unidades. Nível atual do estoque: 130
80: (Rotina) Reabasteceu 50 unidades. Nível atual do estoque: 180
```

SimPy: Gathering KPIs data [SimPy Contributors, 2025]

```
import simpy

t_filas = []
t_sistema = []
uso = {'Medico':0, 'Raio-X':0}
tempo_simulacao = 50

# Processo do paciente
def paciente(env, nome, medico, raioX):
    chegada = env.now
    with medico.request() as req1:
        yield req1
        t_filas.append(env.now - chegada)
        print(f'\tt_filas: {t_filas[-1]:.2f}')
        ini_atend_med = env.now
        yield env.timeout(random.randint(5,10))
        uso['Medico'] += env.now - ini_atend_med
        print(f"\tuso['Medico']: {uso['Medico']:.2f}")
        fim_atend_med = env.now
```

SimPy: Gathering KPIs data [SimPy Contributors, 2025]

```
# Solicita e usa o maquina de Raio-X
with raioX.request() as req2:
    yield req2
    t_filha.append(env.now - fim_atend_med)
    print(f'\tt_filha: {t_filha[-1]:.2f}')
    ini_atend_raioX = env.now
    yield env.timeout(random.randint(3,7)) # Atend Raio-X
    uso['Raio-X'] += env.now - ini_atend_raioX
    print(f"\tuso['Raio-X']: {uso['Raio-X']:.2f}")

# Registra o tempo total gasto no sistema
t_total = env.now - chegada
t_sistema.append(t_total)
print(f'\tt_sistema: {t_sistema[-1]:.2f}')
print(f'{env.now:.2f}: {nome} deixa o hospital. Tempo total
      :{t_total:.2f}')
```

SimPy: Gathering KPIs data [SimPy Contributors, 2025]

```
# Adiciona pacientes ao sistema
def gera_pacientes(env, medico, raioX):
    for i in range(1,6): # Simula 5 pacientes
        env.process(paciente(env, f'Pac {i}', medico, raioX))
        yield env.timeout(0) # Mudar para 10 (analise)

env = simpy.Environment()
medico = simpy.Resource(env, capacity=1)
raioX = simpy.Resource(env, capacity=1)
env.process(gera_pacientes(env, medico, raioX))
env.run(until=tempo_simulacao)

print(f"\n---- Estatisticas KPIs ----")
print(f"Tempo medio em fila:\t{sum(t_fila)/len(t_fila):.2f}")
print(f"Tempo medio sistema:\t{sum(t_sistema)/len(t_sistema)}")
print(f"Utilizacao Medico:\t{uso['Medico']/tempo_simulacao %}")
print(f"Utilizacao Raio-X:\t{uso['Raio-X']/tempo_simulacao %}")
```

SimPy: Gathering KPIs data

```
(venv) PS C:\Users\user\Desktop\sfs\08> py .\ex4-dados.py
0.00: Paciente 1 chega no hospital. Tempo_fila para atendimento medico: 0.00
      utilizacao['Medico'] pelo Paciente 1: 5.00
      tempo_fila de Paciente 1 para exame Raio-X: 0.00
0.00: Paciente 2 chega no hospital. Tempo_fila para atendimento medico: 5.00
      utilizacao['Raio-X'] pelo Paciente 1: 3.00
      tempo_sistema de Paciente 1: 8.00
8.00: Paciente 1 deixa o hospital. Tempo total: 8.00
      utilizacao['Medico'] pelo Paciente 2: 12.00
      tempo_fila de Paciente 2 para exame Raio-X: 0.00
0.00: Paciente 3 chega no hospital. Tempo_fila para atendimento medico: 12.00
      utilizacao['Raio-X'] pelo Paciente 2: 7.00
      tempo_sistema de Paciente 2: 16.00
16.00: Paciente 2 deixa o hospital. Tempo total: 16.00
      utilizacao['Medico'] pelo Paciente 3: 18.00
      tempo_fila de Paciente 3 para exame Raio-X: 0.00
0.00: Paciente 4 chega no hospital. Tempo_fila para atendimento medico: 18.00
      utilizacao['Raio-X'] pelo Paciente 3: 13.00
      tempo_sistema de Paciente 3: 24.00
24.00: Paciente 3 deixa o hospital. Tempo total: 24.00
```

```
24.00: Paciente 3 deixa o hospital. Tempo total: 24.00
      utilizacao['Medico'] pelo Paciente 4: 35.00
      tempo_fila de Paciente 4 para exame Raio-X: 0.00
0.00: Paciente 5 chega no hospital. Tempo_fila para atendimento medico: 25.00
      utilizacao['Raio-X'] pelo Paciente 4: 19.00
      tempo_sistema de Paciente 4: 31.00
31.00: Paciente 4 deixa o hospital. Tempo total: 31.00
      utilizacao['Medico'] pelo Paciente 5: 35.00
      tempo_fila de Paciente 5 para exame Raio-X: 0.00
      utilizacao['Raio-X'] pelo Paciente 5: 26.00
      tempo_sistema de Paciente 5: 42.00
42.00: Paciente 5 deixa o hospital. Tempo total: 42.00

---- Estatísticas Finais ----
-----
Tempo médio em fila:      6.00
Tempo médio no sistema: 24.20
Utilização Médico:      70.00%
Utilização Raio-X:      52.00%
-----
```


SimPy: Class - Advanced data collection [\[SimPy Contributors, 2025\]](#)

```
import simpy
import pandas as pd

class Paciente:
    def __init__(self, idPaciente):
        self.idPaciente = idPaciente

    def __str__(self):
        return f"Paciente {self.idPaciente}"

class RastreiaEvento:
    def __init__(self):
        self.events = []

    def logEvento(self, estagio, evento, nome, reg_tempo):
        self.events.append({
            'Estagio': estagio,
            'Evento': evento,
            'Entidade': nome,
            'Tempo': reg_tempo})
```

SimPy: Class - Advanced data collection [SimPy Contributors, 2025]

```
rastreador = RastreiaEvento() # Exemplo de uso

def atividadeA(env, rastreador):
    j=0
    estagios=['Chegada', 'Triagem']
    for i in range(len(estagios)):
        j+=1
        pac=Paciente(str(j)) # Cria objeto Paciente
        chegada = env.now
        rastreador.logEvento(estagios[i],"Chegada", pac, chegada
        )
        yield env.timeout(5) # Simula o tempo de atendimento
        saida = env.now
        rastreador.logEvento(estagios[i],"Saida", pac, saida)

env = simpy.Environment()
env.process(atividadeA(env, rastreador))
env.run()
```

SimPy: Class - Advanced data collection [SimPy Contributors, 2025]

```
df = pd.DataFrame(rastreador.events) # Cria dataframe do Pandas
df.to_csv('dados_simulacao.csv', index=False) # Exporta csv
dados_triagem = df[df['Estagio']=='Triagem'] # Filtra df
tempo_medio = df['Tempo'].mean() # Calcula media

# Imprime dados
print(f"Eventos: {df}")
print()
print(f"Tempo medio: {tempo_medio:.2f}")
print()
print(f"Triagem: {dados_triagem}")
```

SimPy: Class - Advanced data collection [SimPy Contributors, 2025]

```
(venv) PS C:\Users\user\Desktop\s1s\08> py .\ex6-class-advanced-data-collection.py
```

Eventos:	Estagio	Evento	Entidade	Tempo
0	Chegada	Chegada	Paciente 1	0
1	Chegada	Saída	Paciente 1	5
2	Triagem	Chegada	Paciente 2	5
3	Triagem	Saída	Paciente 2	10

Tempo médio: 5.00

Triagem:	Estagio	Evento	Entidade	Tempo
2	Triagem	Chegada	Paciente 2	5
3	Triagem	Saída	Paciente 2	10

SimPy: Monte Carlo [SimPy Contributors, 2025]

```
import simpy, random, numpy as np
resultados = []

def atend(env, medico):
    inicio = env.now
    with medico.request() as req:
        yield req
        yield env.timeout(random.uniform(5,15))
        tempo_total = env.now - inicio
    return tempo_total

def roda_simulacao(env, medico, num_rodadas): # Simulador
    for _ in range(num_rodadas):
        tempo_total = yield env.process(atend(env,medico))
        resultados.append(tempo_total)

env = simpy.Environment()
medico = simpy.Resource(env, capacity=1)
env.process(roda_simulacao(env, medico, 100)) #100 replicacoes
env.run() #Media:{np.mean(resultados)}, DP:{np.std(resultados)}
```

SimPy: Monte Carlo [SimPy Contributors, 2025]

```
(venv) PS C:\Users\user\Desktop\s1s\08> py .\ex7-montecarlo.py  
Tempo médio de atendimento: 9.79  
Desv. Padrão de atendimento: 2.68
```

SimPy: Factorial Analysis [SimPy Contributors, 2025]

```
import simpy, numpy as np, pandas as pd, itertools

def paciente(env, id_paciente, medico, chegada,
             tempo_atendimento, dado):
    momento_chegada = chegada
    with medico.request() as req:
        yield req
        inicio_atendimento = env.now
        tempo_em_fila = inicio_atendimento - momento_chegada
        yield env.timeout(np.random.exponential(
            tempo_atendimento))
        fim_atendimento = env.now

    dado['Id Paciente'].append(id_paciente)
    dado['Chegada'].append(momento_chegada)
    dado['Inicio atendimento'].append(inicio_atendimento)
    dado['Tempo em fila'].append(tempo_em_fila)
    dado['Fim do atendimento'].append(fim_atendimento)
```

SimPy: Factorial Analysis [SimPy Contributors, 2025]

```
def processo_chegada(env, taxa_chegada, tempo_atendimento,
                    medico, capacidade, dado):
    contador_paciente = 0
    while True:
        interv_chegada = np.random.exponential(1.0/taxa_chegada
        )
        yield env.timeout(interv_chegada)
        contador_paciente += 1
        chegada = env.now

        if medico.count + len(medico.queue) < capacidade:
            env.process(paciente(env, contador_paciente, medico
                                , chegada, tempo_atendimento, dado))
        else:
            dado['Atendimentos perdidos'] += 1
```


SimPy: Factorial Analysis [SimPy Contributors, 2025]

```
def roda_simulacao(taxa_chegada, tempo_atendimento,
                   capacidade_fila, duracao=50):
    env = simpy.Environment()
    medico = simpy.Resource(env, capacity=1)

    dado = {
        "Id Paciente": [],
        "Chegada": [],
        "Inicio atendimento": [],
        "Tempo em fila": [],
        "Fim do atendimento": [],
        "Atendimentos perdidos": 0
    }

    env.process(processo_chegada(env, taxa_chegada,
                                tempo_atendimento, medico, capacidade_fila, dado))
    env.run(until=duracao)
```

SimPy: Factorial Analysis [SimPy Contributors, 2025]

```
atendimentos_finalizados = len(dado['Id Paciente'])
taxa_atendimento = atendimentos_finalizados/duracao
if atendimentos_finalizados > 0:
    tempo_medio_em_fila = np.mean(dado['Tempo em fila'])
    tempo_no_sistema = [f - c for f,c in zip(dado['Fim do
        atendimento'], dado['Inicio atendimento'])]
    tempo_medio_no_sistema = np.mean(tempo_no_sistema)
else:
    tempo_medio_em_fila = 0
    tempo_medio_no_sistema = 0

return {
    "Taxa de chegada": taxa_chegada,
    "Tempo de atendimento": tempo_atendimento,
    "Capacidade da fila": capacidade_fila,
    "Taxa de atendimento": taxa_atendimento,
    "Tempo medio em fila": tempo_medio_em_fila,
    "Tempo medio no sistema": tempo_medio_no_sistema,
    "Atendimentos perdidos": dado["Atendimentos perdidos"]
} # Fim "roda_simulacao"
```

SimPy: Factorial Analysis [SimPy Contributors, 2025]

```
taxa_chegada = [0.8, 1.0, 1.2] # Def valores p/ parametros
tempo_atendimento = [10, 15] # Def valores p/ parametros
capacidade_fila = [2, 5] # Def valores p/ parametros

##### RODA TODAS AS COMBINACOES #####
TEMPO_SIM = 50
resultados = []
for tx_chegada, tmp_atend, cap_fila in itertools.product(
    taxa_chegada, tempo_atendimento, capacidade_fila):
    saida = roda_simulacao(tx_chegada, tmp_atend, cap_fila,
        duracao=TEMPO_SIM)
    resultados.append(saida)

df = pd.DataFrame(resultados) # Poe tudo em um DataFrame
print("Resultados do Experimento:")
print(df)
# Analise Multivariada - Correlacao
matriz_correlacao = df.corr(numeric_only=True)
print("Matriz de Correlacao:")
print(matriz_correlacao)
```

SimPy:Factorial Analysis [SimPy Contributors, 2025]

```
• (venv) PS C:\Users\user\Desktop\s1s\08> py .\ex8-factorial-analysis.py
```

Resultados do Experimento:

	Taxa de chegada	Tempo de atendimento	Capacidade da fila	Taxa de atendimento	Tempo medio em fila	Tempo medio no sistema	Atendimentos perdidos
0	0.8	10	2	0.08	7.749362	9.544254	25
1	0.8	10	5	0.22	6.919531	3.045443	25
2	0.8	15	2	0.02	0.000000	19.712629	38
3	0.8	15	5	0.12	4.667254	7.088121	26
4	1.0	10	2	0.06	9.240060	16.197289	57
5	1.0	10	5	0.24	8.445744	3.885828	31
6	1.0	15	2	0.02	0.000000	13.584698	45
7	1.0	15	5	0.10	18.542051	9.162890	35
8	1.2	10	2	0.06	13.639604	14.774882	59
9	1.2	10	5	0.10	12.699172	9.671712	38
10	1.2	15	2	0.08	6.224030	8.216128	64
11	1.2	15	5	0.06	17.325974	11.043228	50

Matriz de Correlação:

	Taxa de chegada	Tempo de atendimento	Capacidade da fila	Taxa de atendimento	Tempo medio em fila	Tempo medio no sistema	Atendimentos perdidos
Taxa de chegada	1.000000e+00	9.064933e-17	-9.442639e-17	-0.215683	0.547136	0.093731	0.748921
Tempo de atendimento	9.064933e-17	1.000000e+00	3.947460e-17	-0.452839	-0.174499	0.207279	0.144993
Capacidade da fila	-9.442639e-17	3.947460e-17	1.000000e+00	0.654101	0.464194	-0.676241	-0.523235
Taxa de atendimento	-2.156826e-01	-4.528391e-01	6.541010e-01	1.000000	0.125785	-0.864259	-0.528757
Tempo medio em fila	5.471359e-01	-1.744990e-01	4.641935e-01	0.125785	1.000000	-0.184509	0.160535
Tempo medio no sistema	9.373057e-02	2.072789e-01	-6.762407e-01	-0.864259	-0.184509	1.000000	0.493102
Atendimentos perdidos	7.489213e-01	1.449929e-01	-5.232351e-01	-0.528757	0.160535	0.493102	1.000000

SimPy: Architecture 1 [SimPy Contributors, 2025]

```
import simpy

def processo_paciente(env, id_paciente, t_atend, t_exam_img):
    yield env.timeout(t_atend)
    yield env.timeout(t_exam_img)

env = simpy.Environment()
env.process(processo_paciente(env, 1, t_atend=5, t_exam_img=2))
env.process(processo_paciente(env, 2, t_atend=7, t_exam_img=3))
env.run(until=20)
```

SimPy: Architecture 2 [SimPy Contributors, 2025]

```
import simpy

class Paciente:
    def __init__(self, env, id_paciente, t_atend, t_exam_img):
        self.env = env
        self.id_paciente = id_paciente
        self.t_atend = t_atend
        self.t_exam_img = t_exam_img
        self.acao = env.process(self.run()) # Inicia o processo

    def run(self):
        print(f"{self.env.now}: Paciente {self.id_paciente}
              inicia atendimento.")
        yield self.env.timeout(self.t_atend)
        yield self.env.timeout(self.t_exam_img)

env = simpy.Environment()
paciente_1 = Paciente(env, 1, t_atend=5, t_exam_img=2)
paciente_2 = Paciente(env, 2, t_atend=7, t_exam_img=3)
env.run(until=20)
```

SimPy: Architecture 3 [SimPy Contributors, 2025]

```
class Paciente:
    proximo_id = 1
    def __init__(self, env, t_atend, t_exam_img):
        self.env = env
        self.id = Paciente.proximo_id
        Paciente.proximo_id += 1
        self.state = 'Aguardando atend. medico' #Estado inicial
        self.tempo_registro = round(np.random.uniform(2,7),0)
        self.t_atend = t_atend
        self.t_exam_img = t_exam_img
        self.env.process(self.proc_atend()) # Chama o processo

    def proc_atend(self):
        print(f"{self.env.now}:Paciente{self.id}:{self.state}")
        yield self.env.timeout(self.tempo_registro)
        self.state = 'Recebendo atend. medico' #Estado alterado
        print(f"{self.env.now}:Paciente{self.id} {self.state}")
        yield self.env.timeout(self.t_atend)
        self.state = 'Finalizou atend medico' #Estado alterado
        print(f"{self.env.now}:Paciente{self.id} {self.state}")
        self.env.process(self.proc_exam_img())
```

```
def processo_imagem(self):
    self.state = 'Aguard. exame de img' # Estado inicial
    print(f"{self.env.now}:Paciente{self.id}:{self.state}")
    yield self.env.timeout(self.tempo_registro)
    self.state = 'Fazendo o exame img' # Estado alterado.
    print(f"{self.env.now}:Paciente{self.id}:{self.state}")
    yield self.env.timeout(self.t_exam_img)
    self.state = 'Finalizou o exame img' #Estado alterado
    print(f"{self.env.now}:Pac. {self.id} {self.state}.")

env = simpy.Environment()
paciente_1 = Paciente(env, t_atend=5, t_exam_img=2)
paciente_2 = Paciente(env, t_atend=7, t_exam_img=3)
env.run(until=25)
```


SimPy:Factorial Analysis [SimPy Contributors, 2025]

```
(venv) PS C:\Users\user\Desktop\sls\08> py .\ex9-sim-architecture.py
0: Paciente 1 inicia atendimento.
0: Paciente 2 inicia atendimento.
5: Paciente 1 chega ao departamento de imagem.
7: Paciente 2 chega ao departamento de imagem.
7: Paciente 1 finalizou o exame de imagem.
10: Paciente 2 finalizou o exame de imagem.
• (venv) PS C:\Users\user\Desktop\sls\08> py .\ex10-sim-architecture.py
0: Paciente 1 inicia atendimento.
0: Paciente 2 inicia atendimento.
5: Paciente 1 chega ao departamento de imagem.
7: Paciente 2 chega ao departamento de imagem.
7: Paciente 1 finalizou o exame de imagem.
10: Paciente 2 finalizou o exame de imagem.
• (venv) PS C:\Users\user\Desktop\sls\08> py .\ex11-sim-architecture.py
0: Paciente 1 está Aguardando pelo atendimento médico.
0: Paciente 2 está Aguardando pelo atendimento médico.
5.0: Paciente 2 está Recebendo o atendimento médico.
6.0: Paciente 1 está Recebendo o atendimento médico.
11.0: Paciente 1 Finalizou o atendimento médico.
11.0: Paciente 1 está Aguardando o exame de imagem.
12.0: Paciente 2 Finalizou o atendimento médico.
12.0: Paciente 2 está Aguardando o exame de imagem.
17.0: Paciente 1 está Fazendo o o exame de imagem.
17.0: Paciente 2 está Fazendo o o exame de imagem.
19.0: Paciente 1 Finalizou o exame de imagem.
20.0: Paciente 2 Finalizou o exame de imagem.
```

Outline of this lecture

ACD Simpy Models

SimPy Project

SimPy Project: Present your project model in SimPy

Presentation: The group should **present** their **project** model in Simpy with all simulation technologies learned.

Following, the group should **improve** the simulation project model using the SimPy features listed in this class and, if necessary, redesign the ACD model.

References



SimPy Contributors (2025).

Simpy – discrete-event simulation for python.

<https://simpy.readthedocs.io/en/latest/index.html>.